Báo cáo xây dựng website Hôm nay ăn gì

Thông tin báo cáo

Đồ án Advanced Web Design · Nhóm 3. Ngày tổng hợp 07/11/2026. Phiên bản mã nguồn đối chiếu là commit 87a232a trên branch Update-web-6.0 trong folder hom_nay_an_gi-1. Website triển khai tại https://hom-nay-an-gi-nhom3.vercel.app.

Báo cáo trình bày mục tiêu, giao diện, cách hoạt động của các chức năng, tổ chức mã nguồn MVC, thiết kế MySQL, đăng nhập quản trị, quy trình triển khai và kết quả kiểm thử. Phụ lục trích code thật từ phiên bản trên; gói mã nguồn đi kèm dùng để đọc đầy đủ các file được tham chiếu. Không có mật khẩu hay cấu hình truy cập database thật trong báo cáo và gói mã nguồn.

Website đã đáp ứng phạm vi bản đồ án đã thống nhất: gợi ý món ăn, tìm quán liên quan, đọc công thức và quản trị dữ liệu thật. Những phần như tài khoản khách đồng bộ yêu thích, đặt hàng và điều phối shipper chưa triển khai. Vì vậy có thể xem đây là bản hoàn thành trong phạm vi đồ án hiện tại, không phải hệ thống giao đồ ăn hoàn chỉnh.

1 Mục tiêu và phạm vi

1.1 Vấn đề cần giải quyết

Người dùng thường khó quyết định ăn gì, muốn tìm món hợp bữa hoặc chế độ ăn, sau đó cần biết có thể tự nấu hay tìm quán nào có món đó. Website tạo một luồng ngắn: chọn điều kiện, quay món, xem thông tin món và tiếp tục đến công thức hoặc quán liên quan. Thông tin quán và công thức được quản lý tập trung để có thể bổ sung sau khi đưa website lên Internet.

1.2 Hai nhóm người sử dụng

- Khách truy cập xem danh mục, lọc và quay món, đọc công thức, tìm địa điểm, lưu yêu thích trên trình duyệt và mở liên kết ngoài.
- Quản trị viên đăng nhập bằng tài khoản đã được cấp, quản lý món, quán, liên kết, công thức và căn cứ nhãn. Người truy cập bình thường không được phép ghi các danh mục này.

1.3 Quy mô dữ liệu của phiên bản đã kiểm tra

Danh mục cơ sở gồm 105 món ăn, bánh và đồ uống; 105 địa điểm tại Đà Nẵng; 7 nhóm ẩm thực; 14 công thức; 109 liên kết món với quán. Đây là số liệu của bản đã đối chiếu khi kiểm thử ngày 06/11/2026, không phải chỉ số tự cập nhật trong báo cáo. Các thay đổi admin về sau có thể làm số lượng khác đi.

105 địa điểm ban đầu được chuyển đổi từ danh sách Excel do người dùng cung cấp. Sau quá trình chuẩn hóa và nhập, website đang chạy đọc dữ liệu MySQL. Tệp Excel không phải nguồn dữ liệu được đọc trực tiếp ở mỗi lần khách mở trang. Ảnh minh họa được phân biệt với ảnh thật của quán; chưa biết thông tin thì để trống hoặc hiển thị trạng thái chưa có thông tin.

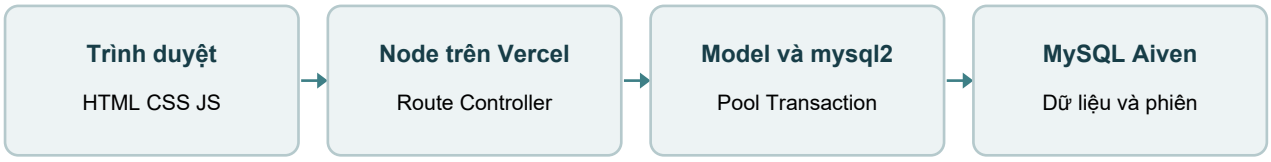
2 Công nghệ và cách tổ chức MVC

2.1 Công nghệ sử dụng

- HTML xây dựng cấu trúc trang; CSS tạo bố cục responsive, màu sắc, thẻ thông tin, modal và chuyển động.
- JavaScript phía trình duyệt xử lý DOM, bộ lọc, quay món, khẩu phần, đánh dấu bước nấu và lưu trữ cục bộ.
- Node.js 24.x chạy backend HTTP bằng module node:http. Dự án không sử dụng Express và không sử dụng EJS.
- MySQL 8.4 LTS lưu dữ liệu có quan hệ; mysql2/promise cung cấp pool và truy vấn bất đồng bộ với tham số.
- Better Auth xử lý xác thực email và mật khẩu, phiên đăng nhập và giới hạn tần suất.
- Aiven cung cấp MySQL online với kết nối TLS kiểm tra chứng chỉ CA. Vercel chạy website và Node Function. GitHub lưu code, schema, migration và seed.

Phiên bản package được khóa trong package-lock.json. package.json của bản đối chiếu khai báo mysql2 ^3.24.6, better-auth ^1.7.16 và @vercel/functions ^3.10.6. Không nên suy ra rằng đây luôn là phiên bản mới nhất của các thư viện.

2.2 MVC trong dự án không dùng EJS



Dữ liệu trả về theo chiều ngược lại để cập nhật giao diện

View là các file HTML/CSS/JS trong public. Controller nhận thông tin yêu cầu, kiểm tra định dạng tham số và điều phối việc gọi Model. Model truy vấn MySQL, ánh xạ bản ghi sang dữ liệu frontend hoặc ghi nhiều bảng trong transaction. Route xác định URL và phương thức HTTP để chuyển yêu cầu đến đúng controller. MVC không bắt buộc View phải là EJS; trong dự án này trình duyệt dựng giao diện từ HTML và dữ liệu do backend cung cấp.

Luồng đọc cơ bản là Trình duyệt → Route → Controller → Model → mysql2 → MySQL → JSON → giao diện. Với dữ liệu FOOD và RECIPES, Node trả JavaScript được sinh từ database để các script frontend hiện có tiếp tục dùng cùng tên biến.

2.3 Cấu trúc thư mục

```
api/index.js                Điểm vào Node Function trên Vercel
public/                     View và tài nguyên gửi xuống trình duyệt
  *.html                    Các trang khách và quản trị
  css/                      Giao diện và responsive
  js/                       Tương tác, API client và bộ lọc
  images/                   Logo, cờ, placeholder và ảnh
server/app.js               Bộ xử lý HTTP và file tĩnh có kiểm soát
server/routes/              Phân luồng URL và phương thức
server/controllers/         Điều phối yêu cầu
server/models/              Đọc và ghi MySQL
server/config/              Cấu hình database và Better Auth
server/middleware/          Xác thực và phân quyền
server/validators/          Ràng buộc dữ liệu đầu vào
server/services/            Tác vụ như cấp tài khoản admin
server/utils/               JSON, Origin và xử lý IP
database/schema.sql         Cấu trúc database ban đầu
database/migrations/        Các thay đổi schema có phiên bản
data/                       Nguồn seed đã chuẩn hóa
scripts/                    Nhập dữ liệu, test và build
tests/                      Kiểm thử tự động
docs/                       Hướng dẫn phát triển và vận hành
vercel.json                 Build, đóng gói Function và route
```

server/views được dành sẵn cho trường hợp cần template về sau; View đang sử dụng thực tế vẫn là public. node_modules là thư viện được cài từ npm, không phải phần mã nguồn tự viết để nộp hoặc commit.

3 Các trang và trải nghiệm người dùng

3.1 Trang chủ và lựa chọn món

index.html chào người dùng theo giờ và cung cấp nút Bất ngờ đi, lựa chọn theo ẩm thực và khu vực Tùy chọn thu gọn. Tùy chọn gồm chế độ ăn, Có công thức, Có quán và số món khớp. Người dùng có thể bắt đầu nhanh hoặc giới hạn tập món trước khi quay. Trạng thái không có món phù hợp được thể hiện rõ; hệ thống không tự bỏ bộ lọc để trả một món ngoài yêu cầu.

cuisines.html hiển thị các nền ẩm thực và lối vào popup quay bất ngờ dùng chung logic điều kiện. Giao diện dùng biểu tượng, cờ và màu tương ứng, có hỗ trợ giảm chuyển động theo cài đặt thiết bị.

3.2 Trang kết quả quay

result.html đọc điều kiện trên URL, xác định tập món hợp lệ và chạy hoạt ảnh đổi tên cùng hình ảnh món. Sau khi dừng, trang hiện tên, mô tả, quốc gia kèm cờ, thẻ bữa ăn và nhãn ăn uống đủ điều kiện. Các hành động gồm quay lại, tìm trên Google Maps, mở lựa chọn nền tảng giao đồ ăn và xem công thức.

Danh sách quán phù hợp được tải và đặt ngay dưới kết quả. Tên món giống nhau không đủ để kết luận mọi quán đều bán; bộ dữ liệu quan hệ quyết định món có liên kết chính xác hay chỉ có biến thể liên quan. Nhãn và trạng thái thiếu công thức không được thay bằng nội dung suy đoán.

3.3 Danh mục món và quán

dishes.html có tìm theo tên, lọc ẩm thực, bữa ăn, chế độ ăn, Món chay, Có quán và Có công thức. Các thẻ món dẫn về thông tin món hoặc công thức, có nút trái tim và các thẻ bữa ăn có icon. restaurants.html tìm theo dữ liệu quán, lọc nhóm món, bữa và khoảng giá, phân trang kết quả từ API. Các dropdown có giao diện đồng bộ thay vì chỉ dùng hình thức mặc định của trình duyệt.

restaurant.html hiển thị thông tin một địa điểm, địa chỉ, nhóm món, giá tham khảo, bữa gợi ý và những thông tin bổ sung đã có. Khoảng giá của quán không được xem là giá xác nhận của một món cụ thể. Các trường chưa có nguồn không tự điền giờ mở cửa, số điện thoại hoặc tình trạng hợp tác.

3.4 Công thức và yêu thích

recipe.html trình bày ảnh, thông tin món, thời gian chuẩn bị và chế biến, nguyên liệu, các bước, mẹo và nguồn tham khảo. Người dùng chọn 1–12 người ăn; nguyên liệu dạng số hoặc khoảng số được tính theo tỷ lệ. Lượng dạng văn bản như một ít giữ nguyên. Thời gian nấu không tự tăng theo tỷ lệ khẩu phần. Checkbox nguyên liệu và bước nấu cập nhật số lượng hoàn thành và thanh tiến độ trong trang.

favorites.html đọc danh sách ID món và quán từ localStorage. Khách không cần đăng nhập để lưu trên thiết bị hiện tại. Các tab cùng origin có thể cập nhật trạng thái qua sự kiện storage. Đổi trình duyệt, xóa dữ liệu trang hoặc đổi tên miền sẽ không tự mang danh sách cũ theo. Không có API đồng bộ yêu thích vào tài khoản khách ở phiên bản này.

3.5 Giao diện sáng tối và hỗ trợ thao tác

Nút icon mở lựa chọn Sáng, Tối, Theo hệ thống; lựa chọn được lưu cục bộ. Chế độ hệ thống phản ứng với prefers-color-scheme. Hiệu ứng chuyển màu, icon và quay món có xét prefers-reduced-motion. Các trang có nhãn điều khiển, trạng thái disabled, skip link, xử lý bàn phím và bố cục trên màn hình nhỏ. Những hỗ trợ này cải thiện khả năng sử dụng; báo cáo không xem chúng là chứng nhận đạt một tiêu chuẩn accessibility toàn diện.

4 Thuật toán quay món và bộ lọc

4.1 Tập món hợp lệ

SpinFilters.parse đọc diet, recipe và linked từ query string. Diet thuộc các giá trị được whitelist; recipe=1 yêu cầu có công thức; linked=1 yêu cầu có quán liên kết chính xác. SpinFilters.apply dùng chung cho số món hiển thị và tập món thực sự được quay. Vì dùng cùng logic, số lượng trên giao diện và kết quả quay không được tính theo hai tiêu chí khác nhau.

Sau khi lọc, App.pickDish ưu tiên món phù hợp bữa hiện tại và không trùng món vừa ra. Nếu không có món đạt cả hai, hàm chọn từ tập đã lọc nhưng khác món trước; cuối cùng mới dùng toàn bộ tập đã lọc. Việc ưu tiên bữa không được phép đưa một món bị loại bởi điều kiện chay/công thức/quán quay trở lại.

4.2 Giờ và tính ngẫu nhiên

Giờ thiết bị được chia thành Sáng từ 05:00 đến trước 11:00; Trưa từ 11:00 đến trước 13:00; Chiều từ 13:00 đến trước 18:00; Tối từ 18:00 đến trước 22:00; các giờ còn lại là Khuya. Món trước được lưu bằng sessionStorage. Chọn phần tử dùng Math.random với chỉ số làm tròn xuống. Đây là gợi ý ngẫu nhiên đơn giản có ưu tiên, không phải AI cá nhân hóa hay thuật toán chứng minh phân phối công bằng cho toàn bộ danh mục.

4.3 Hoạt ảnh và URL

Hoạt ảnh là lớp trải nghiệm trước khi chốt kết quả. Điều kiện và mã món đi qua URL để tải lại hoặc chia sẻ. Các thành phần khóa thao tác phù hợp trong lúc quay; khi hết món, trang hiển thị trạng thái rỗng. Thông tin lọc trong URL không tạo bản ghi database và không phải một cơ chế phân quyền.

5 Quán liên quan và nền tảng bên ngoài

5.1 Liên kết món với quán

restaurant_dishes là bảng nối nhiều món với nhiều quán. Phân loại dish nghĩa là bán đúng món được đối chiếu; related là biến thể hoặc món gần nhau và cần tách biệt khi hiển thị. Liên kết có tên món trong thực đơn, nguồn, căn cứ đối chiếu và giá mỗi phần nếu đã có. Quán hoặc món bị ẩn không được xuất hiện trong gợi ý công khai. Liên kết ngừng công bố được giữ lại trong admin để có thể khôi phục.

API GET /api/dishes/:id/restaurants cấp dữ liệu quán theo món. Kết quả dựa vào quan hệ đã lưu, không tự tìm toàn bộ quán trên Google Maps ở mỗi lần quay. Dữ liệu quán cần tiếp tục được kiểm tra với nguồn thật trước khi xem là thông tin kinh doanh hiện hành.

5.2 Google Maps và giao đồ ăn

Nút Maps tạo URL tìm kiếm với tên món hoặc tên và địa chỉ quán được encode. Website không sở hữu database Google Maps và không khẳng định đã trích xuất toàn bộ dữ liệu quán từ dịch vụ đó.

GrabFood và ShopeeFood mở nền tảng bên ngoài. Popup có danh sách 63 tỉnh thành theo địa giới cũ đã chọn trong thiết kế, lưu lựa chọn khu vực và hướng dẫn người dùng chọn địa chỉ trên nền tảng rồi tìm món. Lựa chọn này không thiết lập địa chỉ giao hàng thật của GrabFood/ShopeeFood và không đảm bảo một đường dẫn tìm kiếm theo tỉnh sẽ luôn được các hãng hỗ trợ.

Website chưa tạo đơn, chưa thu tiền, chưa nhận trạng thái giao hàng và chưa điều phối shipper. Nếu mở rộng phần này phải làm rõ quyền tích hợp, API đối tác, quản lý địa chỉ, món/giá thực tế và vòng đời đơn hàng; chỉ có logo hoặc liên kết không tương đương tích hợp đặt hàng.

6 Nhãn ăn chay và dinh dưỡng

Nhãn Ăn chay cần thông tin nguyên liệu, điều kiện áp dụng và nguồn. Phiên bản món theo một nguồn có thể đủ điều kiện, nhưng phiên bản tại quán chưa chắc giống nhau. Mức nguồn tự công bố phải được phân biệt với mức đã rà soát; không suy đoán một món là chay chỉ từ tên hoặc việc không thấy thịt trong ảnh.

Nhãn dinh dưỡng cũng giới hạn theo phiên bản nguồn và khẩu phần có định lượng. Form admin thu thập g hoặc ml, năng lượng, đường, carbohydrate, nguồn và giới hạn áp dụng. Các tiêu chí ít đường và ít năng lượng được kiểm tra theo số liệu chuẩn hóa 100 g hoặc 100 ml trong code. Không biến một nhãn của nguyên liệu riêng thành kết luận dinh dưỡng cho toàn bộ suất ăn.

Popup của nhãn cho xem tiêu chí, khẩu phần, nguyên liệu hoặc số liệu, điều kiện và nguồn thay vì chỉ có một biểu tượng. Phía giao diện có lựa chọn ít tinh bột, nhưng quản trị chưa hỗ trợ công bố nhãn mới này khi chưa có tiêu chí phù hợp. Không gán đại để tăng số lượng kết quả.

Khi công thức được sửa, xác minh chay dạng reviewed liên quan được thu hồi và ghi lịch sử để rà soát lại. Những căn cứ chỉ áp dụng cho nguồn riêng không tự xác nhận công thức vừa sửa. Đây là quản lý phạm vi dữ liệu, không phải tư vấn dinh dưỡng hoặc chứng nhận suất ăn của quán.

7 Thiết kế MySQL và vòng đời dữ liệu

7.1 Nhóm bảng

- Danh mục: meals, cuisines, dishes, restaurants, dish_meals, restaurant_meals, restaurant_dishes.
- Công thức và căn cứ: recipes, recipe_ingredients, recipe_steps, dish_reviews, dish_nutrition_profiles.

- Xác thực: users, auth_accounts, auth_sessions, auth_verifications, auth_rate_limits.
- Nền tảng quản lý: schema_versions, seed_imports, dataset_documents, restaurant_audit, dish_audit, admin_catalog_versions, admin_catalog_audit.
- Dự phòng cho yêu thích tài khoản: favorite_dishes, favorite_restaurants. Có bảng không đồng nghĩa tính năng đồng bộ đã được triển khai.

Tổng cấu trúc có 26 bảng sau các migration của phiên bản này. Migration 006 bổ sung cột is_visible cho món và quán, mặc định hiện đối với dữ liệu cũ, không thêm bảng mới.

7.2 Quan hệ chính

Một nền ẩm thực có nhiều món. Một món có thể phù hợp nhiều bữa qua dish_meals. Một quán có nhiều bữa gợi ý qua restaurant_meals. Món và quán liên kết nhiều-nhiều qua restaurant_dishes. Mỗi món có công thức được quản lý gắn với danh sách nguyên liệu và bước nấu theo thứ tự. Người dùng có nhiều bản ghi phiên và thông tin xác thực. Audit gắn người sửa với bản trước/sau để theo dõi thay đổi.

Foreign key giúp ngăn bản ghi con trỏ đến món/quán không tồn tại. Các cột và ràng buộc chi tiết nằm trong database/schema.sql và các file migration kèm gói nguồn. Dữ liệu có quan hệ được tách bảng để truy vấn và cập nhật có kiểm soát; một số nguồn gốc/căn cứ vẫn dùng JSON để giữ được nội dung có cấu trúc linh hoạt.

7.3 Schema và seed

schema.sql mô tả cấu trúc, không phải bản sao đầy đủ của database production. data chứa nguồn seed được chuẩn hóa; scripts/import-mysql.mjs kiểm tra dữ liệu, áp dụng schema/migration theo chế độ chạy và nhập dữ liệu khởi tạo. db:check kiểm tra nguồn mà không kết nối MySQL. db:setup được dùng khi dựng database riêng hoặc cập nhật cấu trúc theo hướng dẫn, không chạy tự động trong build Vercel.

Database đã sửa trên Aiven không tự xuất ngược vào JSON. Thành viên lấy code từ Git không tự nhận dữ liệu sống của Aiven, tài khoản admin hay mật khẩu. Muốn cùng dữ liệu cần cấu hình kết nối tới môi trường dùng chung được cấp quyền, hoặc nhập bản sao an toàn vào database riêng.

7.4 Đọc dữ liệu động

Model thực thi SELECT bằng pool mysql2 rồi ánh xạ dữ liệu thành đối tượng frontend. GET /js/data.js sinh const FOOD từ MySQL; GET /js/recipes.js sinh const RECIPES. Hai đường dẫn này được xử lý trước phục vụ tĩnh, kể cả alias đường dẫn được chuẩn hóa, tránh để bản seed tĩnh làm lộ mục đã ẩn.

Khi database không kết nối được, API trả 503 và thông báo chung, không tự dùng JSON cũ để tạo cảm giác dữ liệu đã cập nhật. Cách này giữ tính nhất quán với trạng thái ẩn và nội dung quản trị; đổi lại frontend cần có thông báo tải lỗi rõ ràng và cơ chế thử lại phù hợp nếu tiếp tục hoàn thiện.

8 Đăng nhập và phân quyền quản trị

8.1 Cấp admin

Admin là tài khoản website có role=admin trong MySQL, không phải user root của MySQL và không tự có quyền chỉ vì nhập một email bất kỳ. Công cụ npm run admin:create dùng để cấp tài khoản đầu tiên theo quy trình có kiểm soát. Mật khẩu được nhập riêng ở terminal hoặc hộp nhập trên máy; Better Auth quản lý thông tin xác thực và mã băm. Website không cung cấp đăng ký công khai cho khách ở phiên bản này.

8.2 Đăng nhập

Form gửi POST /api/auth/sign-in/email dưới dạng JSON. Server chỉ chấp nhận endpoint, method, content type, kích thước và Origin hợp lệ; sau đó gọi Better Auth. Nếu đúng tài khoản, server đặt cookie phiên. Trên HTTPS cookie dùng Secure, HttpOnly, SameSite=Lax; mật khẩu và token không được lưu vào localStorage. Giao diện nhận trạng thái thành công rồi tải trang admin; lỗi trả thông báo chung, tránh xuất password hash hoặc nội dung truy vấn.

Phiên lưu trong auth_sessions, thời hạn cấu hình 8 giờ và chu kỳ cập nhật 30 phút. Cấu hình chống thử đăng nhập liên tục dùng lưu trữ rate limit trong database, với giới hạn riêng cho endpoint đăng nhập. Tài khoản website có mật khẩu khác với mật khẩu database dùng để Node kết nối Aiven.

8.3 Kiểm tra quyền trên từng request

Middleware lấy phiên từ cookie, đọc lại role từ users trong MySQL rồi mới cho phép truy cập. Nhờ đó việc thay đổi quyền có tác dụng ở yêu cầu sau, không phụ thuộc role do trình duyệt tự gửi. Chưa đăng nhập trả 401 cho API hoặc chuyển trang về đăng nhập. Đã đăng nhập nhưng không có quyền admin trả 403 cho API. Ẩn nút trên giao diện chỉ là trải nghiệm, không thay thế kiểm tra server.

8.4 Đăng xuất

POST /api/auth/sign-out thu hồi phiên phía MySQL và trả cookie phù hợp. Sau khi đăng xuất, mở lại admin.html phải bị chuyển về admin-login.html. Kiểm thử thực tế đã quan sát được ID phiên bị xóa; các phiên khác không bị xóa hàng loạt. Việc đóng tab đơn thuần không tương đương thao tác đăng xuất có thu hồi phiên.

9 Cách admin ghi dữ liệu thật

9.1 Quán và món

Admin mở form, nhập tên, dữ liệu bắt buộc, trường tùy chọn và trạng thái hiển thị. Frontend gửi POST để thêm hoặc PUT để sửa. Controller đọc JSON và validator kiểm tra whitelist trường, độ dài, URL, danh sách bữa và mã yêu cầu. Mã requestId dùng để nhận diện lần ghi; sửa kèm revision hiện tại.

Model lấy một connection, bắt đầu transaction, kiểm tra lại role của người sửa và khóa bản ghi phù hợp. Khi revision không còn khớp, server trả 409 thay vì ghi đè thay đổi của người khác. Khi hợp lệ, Model ghi bản chính, thay liên kết bữa nếu cần, tăng revision và lưu audit trước/sau cùng người thực hiện. Toàn bộ thành công thì commit; lỗi ở bất kỳ bước liên quan thì rollback, cuối cùng release connection.

9.2 Liên kết và công thức

Admin quản lý đúng cặp món-quán và loại đối chiếu. Nguồn và điều kiện cần được ghi rõ; server chỉ kiểm tra dữ liệu có cấu trúc và URL theo yêu cầu, không tự chứng nhận sự thật của một trang nguồn.

Công thức được lưu qua API quản trị, gồm thời gian, khẩu phần gốc, nguyên liệu dạng số/khoảng/văn bản và các bước. Ngừng công bố gỡ khỏi trang công khai và bộ lọc Có công thức nhưng không xóa nội dung quản trị. Mỗi lần lưu đều có kiểm soát phiên bản và lịch sử. Sửa công thức còn xử lý việc thu hồi xác minh chay cần rà soát lại.

9.3 Ẩn hiện và tránh ghi lặp

Ẩn món hoặc quán ghi is_visible=false, không xóa quan hệ, căn cứ và công thức đi kèm. API công khai lọc lại các bảng liên quan, giảm khả năng xuất hiện mục ẩn qua kết quả quay, số đếm, công thức hoặc liên kết. Admin vẫn xem và khôi phục được.

Retry cùng requestId, nội dung và người thực hiện được xử lý để không tạo bản ghi lặp. Dùng lại mã với nội dung khác hoặc gửi revision cũ trả xung đột. Hệ thống chưa cung cấp chức năng xóa cứng quán hàng loạt; thông thường nên ẩn hoặc ngừng công bố thay vì xóa.

9.4 Danh sách tài khoản và bản nháp cũ

Mục Tài khoản đọc danh sách thật có phân trang, chỉ trả các trường được phép; chưa là hệ thống quản lý người dùng đầy đủ với đăng ký, khôi phục mật khẩu hay thay quyền qua giao diện. Các bản nháp localStorage từ giai đoạn frontend được giữ riêng để đối chiếu, không tự trộn lên database và không tự ghi đè danh mục hiện hành.

10 Danh mục API và mã trạng thái

10.1 API công khai

```
GET /api/dishes
GET /api/dishes?cuisine=viet&meal=sang
GET /api/dishes/:id
GET /api/dishes/:id/restaurants
GET /api/restaurants?q=...&page=1&pageSize=12
GET /api/restaurants/:id
GET /js/data.js
GET /js/recipes.js
```

Danh sách quán hỗ trợ tìm kiếm, category, meal, priceTier và phân trang. Giao diện có thêm logic lọc dựa trên dữ liệu món và nhãn. Không nên suy ra tất cả tên bộ lọc frontend đều là query parameter được backend hỗ trợ; cần xem controller tương ứng khi mở rộng API.

10.2 API xác thực và quản trị

```
POST /api/auth/sign-in/email
POST /api/auth/sign-out
GET  /api/auth/me
GET  /api/admin/me
GET  /api/admin/users
GET  /api/admin/restaurants
POST /api/admin/restaurants
PUT  /api/admin/restaurants/:id
GET  /api/admin/dishes
POST /api/admin/dishes
PUT  /api/admin/dishes/:id
GET  /api/admin/links
POST /api/admin/links
PUT  /api/admin/links/:encodedId
GET  /api/admin/recipes
POST /api/admin/recipes
PUT  /api/admin/recipes/:dishId
GET  /api/admin/reviews
PUT  /api/admin/reviews/:dishId
```

ID liên kết là dishId/restaurantId, phải encodeURIComponent khi đưa vào đoạn URL. Căn cứ nhãn được chỉnh cho món có trong danh sách, không cần mặc định coi POST là thao tác dùng để thêm căn cứ mới.

200 nghĩa là đọc hoặc sửa thành công; 201 tạo bản ghi mới; 302 điều hướng trang yêu cầu đăng nhập. 400 dữ liệu/đường dẫn không hợp lệ; 401 chưa xác thực; 403 thiếu quyền hoặc Origin không hợp lệ; 404 không có tài nguyên; 405 sai phương thức; 409 xung đột phiên bản; 413 nội dung quá lớn; 415 sai loại nội dung; 429 vượt tần suất; 503 hệ thống dữ liệu tạm không sẵn sàng.

11 Cấu hình chạy và triển khai

11.1 Chạy trên máy thành viên

```
git clone --branch Update-web-6.0 https://github.com/GabTommy2006/hom_nay_an_gi.git
cd hom_nay_an_gi
npm ci
# Sao chép .env.example thành .env và điền cấu hình riêng
# Tạo database và cấp quyền theo database/README.md
npm run db:check
npm run db:setup
npm start
```

Mở http://127.0.0.1:3000. Nếu dùng gói nguồn ZIP, giải nén rồi chạy npm ci trong thư mục chứa package.json. Cần Node 24.x và MySQL 8.4 hoặc môi trường Aiven được cấp quyền. Không chạy db:setup trên production chỉ để kiểm tra giao diện, và không đưa mật khẩu vào lệnh Git hoặc README.

Các biến quan trọng là DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_CA, BETTER_AUTH_SECRET và BETTER_AUTH_URL. HOST/PORT phục vụ chạy local. .env.example chỉ là mẫu, không phải file có thể dùng ngay để đăng nhập Aiven. Khi chia sẻ dự án cần cung cấp cấu hình qua kênh riêng được quản lý; không chia sẻ mật khẩu root làm tài khoản chung.

11.2 Vercel kết nối Aiven

api/index.js export handleRequest làm Function. vercel.json đưa toàn bộ request qua điểm vào này để các ràng buộc admin và dữ liệu động được giữ đúng. public được đóng gói trong Function; dist là đầu ra build tối thiểu, không phải một bản seed công khai thay thế backend. Build chỉ chuẩn bị đầu ra, không nhập schema hoặc ghi database.

Biến môi trường production nằm trong project Vercel, bao gồm CA và khóa xác thực. BETTER_AUTH_URL phải đúng origin HTTPS đang sử dụng; khi đổi tên miền phải cập nhật và redeploy. NODEJS_HELPERS=0 được dùng để giữ cách đọc raw body của HTTP handler. Không chuyển HOST/PORT local thành cấu hình cổng của Function.

Region bom1 được chọn trong cấu hình triển khai. Vị trí thực thi và độ trễ cần kiểm tra bằng log thực tế, không chỉ dựa vào tên region trong file. Dịch vụ MySQL vẫn nằm trên Aiven; đưa code lên Vercel không đưa database MySQL local đi cùng.

11.3 Sao lưu và quy trình phát hành

Các branch 4.0, 5.0, 6.0 từng được tạo để giữ mốc chuyển backend và quản trị. Branch bảo vệ code, không bảo vệ nội dung MySQL đã thay đổi. Cần backup database riêng trước migration hoặc chức năng ghi quan trọng. Bản sao lưu chứa tài khoản/phiên phải được bảo quản như dữ liệu nhạy cảm, không đẩy vào Git.

Folder hom_nay_an_gi-1 là nguồn được đối chiếu. Ngày 06/11/2026, nội dung đã được chép sang hom_nay_an_gi và giữ Git đích riêng trỏ TonyLikeDev. Lần kiểm tra quyền khi đó chỉ có pull, chưa có push tới repo TonyLikeDev; báo cáo không khẳng định code mới đã được commit lên repo này.

12 Sự cố thực tế và cách khắc phục

12.1 Triệu chứng mất dữ liệu từng lúc

Khi kiểm tra sau đổi tên miền, HTML vẫn trả 200 nhưng /js/data.js, /js/recipes.js và API trả 503. Vì frontend cần các biến FOOD và RECIPES để dựng nội dung, người dùng nhìn thấy khung trang mà không có danh mục. Có lúc kết nối được giải phóng thì dữ liệu xuất hiện lại; nguyên nhân không chỉ là trình duyệt tải chậm.

12.2 Bằng chứng và sửa code

Log Vercel ghi Database request failed: ER_CON_COUNT_ERROR. Lần đọc trạng thái Aiven cho thấy max_connections=76 và Max_used_connections từng đạt 78. Nhiều instance có thể cùng tích lũy kết nối rảnh. Cấu hình trước đặt connectionLimit=5 và maxIdle=5; trong phiên bản mysql2 đang dùng, timer dọn kết nối chỉ được khởi tạo khi maxIdle nhỏ hơn connectionLimit.

Commit 87a232a thay bằng connectionLimit=2, maxIdle=0 và idleTimeout=1000 ms để đóng kết nối rảnh. attachDatabasePool được gắn khi chạy Vercel. Đây là giảm tài nguyên dùng bởi mỗi instance, không phải bảo đảm database không bao giờ hết kết nối khi lượng truy cập tăng mạnh. Nếu mở rộng cần đo tổng instance, tài nguyên Aiven, thời gian truy vấn và chiến lược cache hoặc quản lý kết nối bổ sung.

12.3 Xác minh sau sửa

Test thực thi nhiều truy vấn đồng thời, kiểm tra số connection không quá 2, chờ các connection rảnh về 0 rồi xác nhận request mới kết nối lại được. Bản sửa được build, commit, push và triển khai production. Kiểm tra online 5 lượt với 4 đường dẫn dữ liệu mỗi lượt đều trả 200; trang Ẩm thực đã được quan sát có nội dung trở lại. Đây là kết quả kiểm tra tại thời điểm sửa, không phải chứng nhận độ sẵn sàng dài hạn.

Lần đăng nhập admin mà người dùng mô tả mất một lúc sau đó đã thành công. Log kiểm tra không ghi nhận lỗi server trong khoảng xem; chưa có số đo đầy đủ để kết luận nguyên nhân riêng của độ trễ lần đăng nhập đó. Không nên tự quy mọi trường hợp chậm về cold start hay về Aiven khi chưa có dữ liệu đo.

13 Kiểm thử và kết quả nghiệm thu

13.1 Kiểm thử tự động

npm test dựng MySQL tạm riêng thay vì dùng .env và database Aiven thật. Bộ test kiểm tra import và seed, Model/API, tìm kiếm/phân trang, liên kết món-quán, công thức, căn cứ nhãn, bộ lọc quay, draft cũ, xác thực, phân quyền, CSRF, giới hạn request, phiên, ghi dữ liệu, audit, xung đột revision và rollback. Các test về ẩn/hiện kiểm tra tính nhất quán trên dữ liệu công khai.

Kết quả lần chạy ngày 06/11/2026 sau sửa pool: 30 test được liệt kê, 29 đạt, 0 lỗi và 1 bỏ qua theo cấu hình. Test import sâu có chế độ riêng npm run test:db; báo cáo không coi mục bị bỏ qua là một test đã chạy thành công trong lần npm test đó. npm run build:vercel hoàn tất.

13.2 Kiểm thử trực tiếp trên Vercel và Aiven

Đã kiểm tra HTML, danh mục 105 món, danh sách 105 quán, trang quay, quán liên quan và công thức xôi xoài. Truy cập admin khi chưa đăng nhập được chuyển về trang đăng nhập; API admin trả 401. Đường dẫn /.env trả 404. Đăng nhập thông tin không hợp lệ trả 401 với thông báo chung.

Người dùng tự nhập mật khẩu admin trên Chrome. Sau khi trang quản trị hiện, kiểm tra một quán thử riêng ở trạng thái ẩn: thêm thành công với revision=1; sửa tên, ghi chú và bữa từ Sáng/Trưa sang Tối thành revision=2. Đối chiếu trực tiếp trong Aiven có đủ audit create và update; tải lại admin vẫn thấy nội dung

mới; API công khai trả 404 cho quán thử vì bị ẩn.

Đăng xuất qua giao diện đã xóa đúng phiên đang dùng; mở lại admin chuyển về đăng nhập. Quán thử và dữ liệu phụ thuộc của chính quán thử được dọn sau kiểm tra. Đối chiếu SHA-256 trước/sau trên 105 quán gốc và liên kết bữa xác nhận dữ liệu gốc không đổi. Không dùng tên, địa chỉ hoặc thông tin quán thật để thử ghi đè.

13.3 Kịch bản kiểm tra khi tiếp tục phát triển

- Chạy npm test trước khi phát hành thay đổi backend và db:check khi sửa seed.
- Kiểm tra lại kết quả quay đúng điều kiện, trạng thái không có món và nhãn có nguồn.
- Đăng nhập, tải lại admin, ghi một bản thử riêng, xác nhận database và đăng xuất.
- Khi sửa công thức kiểm tra tỷ lệ khẩu phần, thứ tự bước và thu hồi căn cứ chay cần rà soát.
- Khi ẩn quán/món kiểm tra danh sách, chi tiết, số đếm, công thức và quán liên quan.
- Kiểm tra màn hình nhỏ, bàn phím, chế độ sáng/tối, storage bị chặn và database tạm lỗi.

14 Giới hạn và hướng hoàn thiện

Phạm vi tiếp theo nên ưu tiên độ tin cậy dữ liệu quán, thông báo tải lỗi rõ ràng, đo độ trễ đăng nhập và theo dõi lỗi kết nối. Các bộ dữ liệu seed và nguồn ảnh cần được rà soát định kỳ. Việc website chạy được không xác minh thay cho chủ quán về thực đơn, giá, giờ mở cửa hay địa chỉ giao hàng.

Tài khoản khách và đồng bộ Yêu thích chưa làm. Các bảng favorite_* dành sẵn chưa được nối thành chức năng người dùng. Chưa có đăng ký, khôi phục mật khẩu và bộ công cụ quản trị tài khoản đầy đủ. Chưa có thống kê người truy cập, lượt quay hoặc đơn hàng thật; màn hình tổng quan hiện dùng số lượng danh mục, không giả lập chỉ số vận hành.

Đặt hàng, thanh toán, shipper và hợp tác quán chưa thuộc bản hiện tại. Upload ảnh trực tiếp cũng chưa là quy trình lưu trữ media hoàn chỉnh; admin đang có URL ảnh thay thế được kiểm tra định dạng. Khả năng chịu tải cao, uptime dài hạn và một audit bảo mật độc lập chưa được chứng minh bởi bộ test chức năng hiện có.

15 Hướng dẫn đọc code đi kèm

Bắt đầu từ server/app.js để hiểu request đi đâu, sau đó server/routes, controllers và models theo đúng chức năng. Với giao diện, đọc HTML tương ứng rồi các script gắn cuối trang. public/js/app.js cung cấp hàm dùng chung; spin-filters.js quản lý điều kiện; recipe.js xử lý khẩu phần và tiến độ; favorites.js quản lý ID trên trình duyệt; admin.js và các module liên quan kết nối API quản trị.

database/schema.sql và migrations giải thích cấu trúc hiện hành; scripts/import-mysql.mjs mô tả khởi tạo dữ liệu. tests cho thấy trường hợp hợp lệ, từ chối và rollback. docs/AUTH.md, RESTAURANT-ADMIN.md, DISH-ADMIN.md, ADMIN-CATALOG.md, VISIBILITY.md, VERCEL.md và database/README.md là hướng dẫn chuyên sâu.

Gói ZIP kèm báo cáo chỉ chứa các file được Git theo dõi của commit đối chiếu. Không có .git, node_modules, .env thật, file cấu hình Aiven riêng, backup SQL production hoặc dữ liệu phiên người dùng. .env.example là mẫu an toàn để thành viên tự cấu hình. Mã băm SHA-256 của từng file trong gói được ghi trong MANIFEST-SHA256.txt để đối chiếu.

16 Kết luận

Website đã chuyển từ frontend dùng dữ liệu chuẩn hóa sang mô hình MVC có backend Node.js và MySQL online, giữ được giao diện và luồng gợi ý món. Admin đã đăng nhập thật, phân quyền phía server và lưu các danh mục vào Aiven. Bản production đã triển khai trên Vercel và được kiểm tra các luồng cốt lõi.

Giá trị chính của bản đồ án là kết nối quyết định ăn gì với thông tin món, công thức và quán có căn cứ. Các giới hạn về đặt hàng, đồng bộ tài khoản khách và xác thực thông tin kinh doanh cần được trình bày đúng khi demo. Mã nguồn và cấu trúc hiện tại tạo cơ sở để phát triển tiếp, nhưng không nên mô tả các chức năng dự kiến như đã hoạt động.

17 Nguồn đối chiếu

- Mã nguồn commit 87a232a trên branch Update-web-6.0; gói nguồn và phụ lục code đính kèm.
- README.md, database/README.md và tài liệu trong docs của dự án. Một số đoạn README còn phản ánh các mốc cũ; báo cáo dùng code hiện tại và kết quả kiểm thử để xác định trạng thái chức năng.
- Kết quả npm test, build và kiểm thử online đã thực hiện ngày 06/11/2026 trong quá trình triển khai.
- Vercel Node.js runtime: https://vercel.com/docs/functions/runtimes/node-js
- Vercel quản lý kết nối: https://vercel.com/kb/guide/connection-pooling-with-functions
- Vercel cấu hình Node nâng cao: https://vercel.com/docs/functions/runtimes/node-js/advanced-node-configuration
- Nguồn công thức và dinh dưỡng cụ thể được lưu cùng từng công thức/căn cứ trong dữ liệu, không áp dụng một nguồn chung cho mọi biến thể món.

Phụ lục Code chính và giải thích

Các đoạn tiếp theo được trích tự động từ file thật của phiên bản đối chiếu. Chúng dùng để giải thích cấu trúc và luồng xử lý; các đoạn trích không phải chương trình độc lập để chạy tách khỏi dự án. Xem gói nguồn để lấy phần import, các helper và cấu hình đầy đủ.

A Điểm vào Vercel

File api/index.js, dòng 1 đến 1. Function dùng lại handleRequest của server local để tránh tạo hai bộ phân quyền khác nhau.

```
// Keep raw IncomingMessage/ServerResponse semantics for the existing MVC backend.
```

B Phân luồng HTTP

File server/app.js, dòng 17 đến 30. Auth và admin được xử lý trước các API công khai. Endpoint API lạ trả 404; yêu cầu file tĩnh chỉ hỗ trợ GET và HEAD.

```
export async function handleRequest(req, res) {
  try {
    const url = new URL(req.url, 'http://localhost');
    if (await handleAuth(req, res, url)) return;
    if (await handleAdmin(req, res, url)) return;
    if (await handleDishes(req, res, url)) return;
    if (await handleRestaurants(req, res, url)) return;
    if (url.pathname.startsWith('/api/')) {
      res.writeHead(404, { 'Content-Type': 'application/json; charset=utf-8' });
      return res.end(JSON.stringify({ message: 'API không tồn tại.' }));
    }
    if (!['GET', 'HEAD'].includes(req.method)) {
      res.writeHead(405, { Allow: 'GET, HEAD' });
```

```
      return res.end();
```

C Sinh dữ liệu frontend từ MySQL

File server/app.js, dòng 38 đến 48. FOOD và RECIPES giữ nguyên hợp đồng frontend, nhưng nội dung lấy từ Model MySQL; dữ liệu trả về không được cache thành bản seed cũ.

```
    }
    const fileKey = value => process.platform==='win32'?value.toLowerCase():value;
    const isFood = fileKey(filename)===fileKey(path.join(publicDir,'js','data.js'));
    const isRecipe = fileKey(filename)===fileKey(path.join(publicDir,'js','recipes.js'));
    // Resolve aliases before serving: encoded paths must not expose the static seed snapshots.
    if (isFood || isRecipe) {
      const data = await (isFood ? getFood() : getRecipes());
      res.writeHead(200, { 'Content-Type': mime['.js'], 'Cache-Control':'no-store' });
      // Escape HTML delimiters without changing the synchronous frontend contract.
      const serialized = JSON.stringify(data).replace(/</g, '\\u003c');
      return res.end(req.method === 'HEAD' ? undefined : `const ${isFood ? 'FOOD' : 'RECIPES'} =
    ${serialized};`);
```

D Pool MySQL và giải phóng kết nối

File server/config/database.js, dòng 16 đến 28. Pool dùng lại trong instance. maxIdle nhỏ hơn connectionLimit kích hoạt dọn kết nối rảnh của mysql2. rows truyền tham số tách khỏi SQL.

```
let pool;
export function getPool() {
  if (!pool) {
    // mysql2 only starts its idle cleanup timer when maxIdle < connectionLimit.
    // Release every idle connection so suspended serverless instances do not
    // exhaust the shared database's connection budget.
    try { pool=mysql.createPool({...connectionConfig(),connectionLimit:2,maxIdle:0,idleTimeout:100
    0,waitForConnections:true,queueLimit:100,enableKeepAlive:true});if(process.env.VERCEL==='1')at
    tachDatabasePool(pool.pool); }
    catch { const error=new Error('Database configuration is missing or
    invalid.');error.code='DB_CONFIG_INVALID';throw error; }
  }
  return pool;
}
export async function rows(sql,params=[]) { const [result]=await getPool().execute(sql,params);
    return result; }
export async function closeDatabase(){const current=pool;pool=undefined;if(current)await
    current.end();}
```

E Kiểm tra admin phía server

File server/middleware/auth.js, dòng 5 đến 18. Phiên được Better Auth xác minh; role được đọc lại từ users. API trả 401 hoặc 403; trang HTML điều hướng về đăng nhập.

```
export async function currentUser(req){
 const session=await getAuth().api.getSession({headers:fromNodeHeaders(req.headers||{})});
 if(!session)return null;
 // Always read role from MySQL: demotion applies to the next request, no cookie role cache.
 const users=await rows('SELECT id,email,display_name,role FROM users WHERE
    id=?',[session.user.id]);
 if(!users.length)return null;
 const user=users[0];return
    {id:String(user.id),email:user.email,name:user.display_name,role:user.role};
}
export async function requireAdmin(req,res,{page=false}={}){
 const user=await currentUser(req);
 if(!user){if(page){res.writeHead(302,{'Location':'/admin-login.html','Cache-Control':'no-
    store'});res.end();}else send(res,401,{message:'Vui lòng đăng nhập để tiếp tục.'});return
    null;}
 if(user.role!=='admin'){if(page){res.writeHead(302,{'Location':'/admin-
    login.html?reason=forbidden','Cache-Control':'no-store'});res.end();}else
    send(res,403,{message:'Tài khoản không có quyền quản trị.'});return null;}
 return user;
}
```

F Chọn món trong tập đã lọc

File public/js/app.js, dòng 25 đến 37. Hàm chọn món dùng sessionStorage để giảm lặp và ưu tiên bữa hiện tại. Tập pool do bộ lọc cung cấp, không tự đưa món ngoài tập vào.

```
// Trình duyệt chặn lưu trữ thì chỉ mất tính năng "không lặp món", trang vẫn chạy.
const getLast = () => { try { return sessionStorage.getItem(LAST_KEY); } catch { return null; }
  };
const setLast = (id) => { try { sessionStorage.setItem(LAST_KEY, id); } catch { /* bỏ qua */ }
  };

/** Ưu tiên món hợp bữa hiện tại, không lặp lại món vừa ra. */
function pickDish(pool) {
  const last = getLast();
  const meal = getMeal();
  const fit = pool.filter((d) => d.meals.includes(meal));
  const tiers = [fit.filter((d) => d.id !== last), pool.filter((d) => d.id !== last), pool];
  return randomItem(tiers.find((t) => t.length));
}
```

G Áp dụng tùy chọn quay

File public/js/spin-filters.js, dòng 3 đến 11. Whitelist diet và điều kiện recipe/linked được dùng chung cho số lượng món và lượt quay.

```
const diets = ['all','vegetarian','low-sugar','low-carb','low-energy'];
function parse(params) {
  const diet = params.get('diet') || 'all';
  return { diet: diets.includes(diet) ? diet : 'all', recipe: params.get('recipe') === '1',
  linked: params.get('linked') === '1', invalid: !diets.includes(diet) ||
  ['recipe','linked'].some(key => params.has(key) && !['0','1'].includes(params.get(key))) };
}
const apply = (dishes, options) => dishes.filter(dish =>
  (options.diet === 'vegetarian' ? App.isVegetarian(dish) :
  Nutrition.matches(dish,options.diet)) &&
  (!options.recipe || Object.hasOwn(RECIPES,dish.id)) && (!options.linked ||
  dish.linkedRestaurantCount > 0));
const count = options => Number(options.diet !== 'all') + Number(options.recipe) +
  Number(options.linked);
```

H Tính định lượng nguyên liệu

File public/js/recipe.js, dòng 65 đến 73. Lượng mới bằng lượng gốc nhân số người mới chia khẩu phần gốc. Khoảng số được tính từng đầu mút; văn bản giữ nguyên.

```
const numberFormat = new Intl.NumberFormat('vi-VN', { maximumFractionDigits: 2 });
function formatQuantity(ingredient) {
  if (ingredient.text) return ingredient.text;
  const scaled = value => numberFormat.format(value * servings / recipe.servings);
  const value = Array.isArray(ingredient.amount)
    ? ingredient.amount.map(scaled).join('–') : scaled(ingredient.amount);
  return `${value} ${ingredient.unit}`;
}
function updateServings(announce = false) {
```

I Kiểm tra phiên bản khi ghi quán

File server/models/admin-restaurant.model.js, dòng 6 đến 20. Một connection duy nhất được giữ cho transaction. Khóa bản ghi và revision chống ghi đè; requestId hỗ trợ xử lý lần ghi lặp.

```
async function writeRestaurant({values,revision,requestId},{id,actorId}={}){
 const c=await getPool().getConnection();
 const target=id||'r-'+requestId;
 try{
  await c.beginTransaction();
  const [[actor]]=await c.execute('SELECT role FROM users WHERE id=? FOR SHARE',[actorId]);
  if(actor?.role!=='admin')throw new HttpError(403,'Tài khoản không còn quyền quản trị.');
  const [[current]]=await c.execute('SELECT * FROM restaurants WHERE id=? FOR UPDATE',[target]);
  if(id&&!current)throw new HttpError(404,'Không tìm thấy quán để sửa.');
  if(!id&&current){
   const [[audit]]=await c.execute("SELECT actor_user_id,after_data FROM restaurant_audit WHERE
    restaurant_id=? AND action='create' ORDER BY id LIMIT 1",[target]);
   if(!audit||String(audit.actor_user_id)!==String(actorId)||!isDeepStrictEqual(audit.after_data,v
    alues))throw new HttpError(409,'Yêu cầu này đã được sử dụng. Hãy tải lại danh sách trước khi
```

```
    tạo tiếp.');
   await c.commit();return {id:target,created:false};
  }
  if(current&&current.revision!==revision)throw new HttpError(409,'Quán đã được cập nhật ở nơi
    khác. Hãy đóng form, tải lại trang rồi mở lại để sửa; nội dung đang nhập chưa được lưu.');
```

I2 Lưu liên kết và kết thúc transaction

File server/models/admin-restaurant.model.js, dòng 30 đến 40. Sau phần INSERT hoặc UPDATE bản chính, Model ghi trạng thái hiển thị, bữa và audit, rồi commit. Lỗi dẫn tới rollback; finally luôn release connection.

```
  await c.execute('UPDATE restaurants SET is_visible=? WHERE
    id=?',[values.isVisible??(current?!!current.is_visible:true),target]);
  const [oldMeals]=await c.execute('SELECT meal_id FROM restaurant_meals WHERE restaurant_id=?
    ORDER BY sort_order',[target]);
  if(!isDeepStrictEqual(oldMeals.map(r=>r.meal_id),values.meals)){
   await c.execute('DELETE FROM restaurant_meals WHERE restaurant_id=?',[target]);
   for(const [index,meal] of values.meals.entries())await c.execute('INSERT INTO restaurant_meals
    (restaurant_id,meal_id,sort_order) VALUES (?,?,?)',[target,meal,index+1]);
  }
  await c.execute('INSERT INTO restaurant_audit
    (restaurant_id,actor_user_id,action,before_data,after_data) VALUES
    (?,?,?,?,?)',[target,actorId,id?'update':'create',current?json(current):null,json(values)]);
  await c.commit();
  return {id:target,created:!id};
 }catch(error){await c.rollback();throw error;}finally{c.release();}
}
```

J Kiểm soát JSON và Origin

File server/utils/http.js, dòng 1 đến 10. Không tin dữ liệu từ trình duyệt: giới hạn kích thước, kiểm tra Origin, loại nội dung và cấu trúc JSON trước khi ghi.

```
import {authOrigin} from '../config/auth.js';
export class HttpError extends Error
    {constructor(status,message){super(message);this.status=status;}}
export async function jsonMutation(req,{limit=16384}={}){
 if(req.headers.origin!==authOrigin()||req.headers['sec-fetch-site']==='cross-site')throw new
    HttpError(403,'Yêu cầu không hợp lệ. Hãy thao tác từ website.');
 if(!/^application\/json(?:\s*;|$)/i.test(req.headers['content-type']||''))throw new
    HttpError(415,'Yêu cầu phải có định dạng JSON.');
 let size=0;const chunks=[];
 for await(const chunk of req){size+=chunk.length;if(size>limit)throw new HttpError(413,'Nội dung
    vượt quá dung lượng cho phép.');chunks.push(chunk);}
 let value;try{value=JSON.parse(Buffer.concat(chunks).toString('utf8'));}catch{throw new
    HttpError(400,'Nội dung JSON không hợp lệ.');}
 if(!value||Array.isArray(value)||typeof value!=='object')throw new HttpError(400,'Nội dung gửi
    lên không hợp lệ.');return value;
}
```

K Cấu hình Vercel

File vercel.json, dòng 1 đến 16. Toàn bộ route đi qua Function. includeFiles đóng gói public cho bộ phục vụ tĩnh có kiểm soát; build không nhập dữ liệu.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": null,
  "buildCommand": "npm run build:vercel",
  "outputDirectory": "dist",
  "regions": ["bom1"],
  "functions": {
    "api/index.js": {
      "includeFiles": "public/**",
      "maxDuration": 60
    }
  },
  "routes": [
    {"src": "/(.*)", "dest": "/api/index"}
  ]
}
```